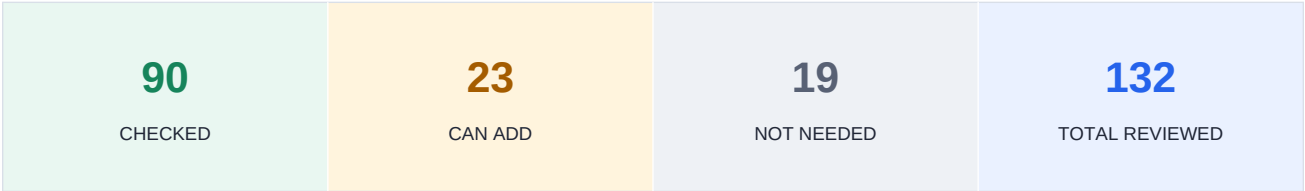


Agentic AI, LLMOps, Cloud Deployment and Privacy

Updated project checklist audit for Bob IDE

A code-evidenced reassessment after implementing authentication, private workspace ownership, role-based developer operations, exhausted-retry DLQ handling, prompt correction, human evaluation, structured telemetry and the Lightning/Colab v4 runtime contract.



Status key

CHECKED - directly evidenced in the current repository. **CAN ADD** - relevant and feasible but absent or incomplete.
NOT NEEDED - outside the current product scope; reconsider if scope changes.

Assessment date: 6 August 2026. Repository: code_editor_ai. Source: the supplied 20-page Module 10 checklist PDF.

Executive reassessment

Bob IDE now satisfies the planned local/demo authentication and governance boundary. The largest gains are server-side identity and ownership enforcement, retry/DLQ reliability, Admin-only quality operations, human evaluation, prompt/model/version telemetry, user feedback and a hardened Lightning/Colab notebook contract.

90 CHECKED	23 CAN ADD	19 NOT NEEDED	132 TOTAL REVIEWED
---------------	---------------	------------------	-----------------------

MEASURE	PREVIOUS AUDIT	UPDATED	CHANGE
Checked	40	90	+50
Can add	72	23	-49
Not needed	20	19	-1

Newly completed controls

- First-run Admin setup, bcrypt password storage, opaque server sessions, CSRF, expiry/revocation and login throttling.
- Admin/User RBAC, owner-only workspaces and authenticated REST, MCP gateway, Socket.IO, LSP and terminal application boundaries.
- Three-attempt transient retry, dlq_pending runs, immutable attempt history and Admin correction/dismissal workflow.
- Failed-review quality records plus user-controlled Force Apply and Stage with a reason and one-time approval token.
- Five-dimension human evaluation with revisions, prompt corrections, exports and a 12-case versioned offline dataset.
- Redacted request/tool/model/auth/audit JSONL, usage/cost/latency summaries and accepted/rejected/force-applied feedback.
- Lightning/Colab v4 contract with mandatory bearer authentication, serialized GPU requests, usage, tracing and safe errors.

Verification

The complete repository command passed: 18 Python tests, 9 Node tests, a 95-frontend/104-Python tool contract, notebook validation, all 12 offline evaluations, frontend lint and production build. Remaining warnings are Fast Refresh organization and bundle size, not test failures.

1. Agentic AI Foundations

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Has a planner	CHECKED	A dedicated plan stage produces a selectable plan before code generation. Evidence: bob_core/model_service.py; capabilities.py
Has at least two tools	CHECKED	The MCP catalogue exposes file, Git, worktree, model, context, test and workspace tools. Evidence: capabilities.py; mcp_server.py
Memory	CHECKED	Plans, runs, model stages, proposals, ownership and workflow artifacts persist in JSON stores. Evidence: bob_core/*_store.py; node-server/auth-store.js
Retry	CHECKED	Transient connection, timeout, HTTP 429 and HTTP 5xx failures retry at most three times with bounded exponential backoff and jitter. Evidence: bob_core/colab_adapter.py
Reflection	CHECKED	Replanning and a distinct reviewer stage inspect intermediate plans and code and return PASS or FAIL. Evidence: bob_core/model_service.py
Human approval	CHECKED	Reviewer-failed proposals are server-blocked from normal apply. Force Apply and Stage requires a reason, confirmation and one-time approval. Evidence: bob_core/proposal_store.py; node-server/server.js
Structured output	CHECKED	MCP and gateway responses use JSON, while external model boundaries use Pydantic v2 contracts. Evidence: bob_core/contracts.py; mcp_server.py
Error handling	CHECKED	Invalid input, model, tool and transport failures become structured errors, failed runs or DLQ records. Evidence: bob_core/model_service.py; node-server/dev-store.js
Logging	CHECKED	Request, model, tool, authentication, approval, evaluation and audit events are persisted with redaction. Evidence: node-server/dev-store.js

Metric coverage: Tool selection and step-efficiency metrics still require labeled expected decisions; reliability and operational outcomes are now captured.

2. LangChain, LangGraph and CrewAI

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Tool abstraction	CHECKED	Capabilities have stable names, generated descriptions, typed inputs and MCP schemas. Evidence: capabilities.py; scripts/verify-api-contract.mjs
Prompt templates	CHECKED	Planner, coder and reviewer templates are reusable and every run carries a prompt-set version. Evidence: Untitled28_lightning_ai_bob_runtime.ipynb; bob_core/model_config.py
State management	CHECKED	Run, plan, proposal, session, DLQ, correction, evaluation and feedback state uses versioned JSON with atomic updates. Evidence: bob_core/json_store.py; node-server/*.js
Retry	CHECKED	Transient Colab nodes use a fixed three-attempt retry policy and preserve attempt history. Evidence: bob_core/colab_adapter.py
Conditional routing	CHECKED	Execution branches through plan/replan, PASS/FAIL review, conflict handling, retry exhaustion and DLQ resolution. Evidence: bob_core/model_service.py; node-server/dev-store.js
Human node	CHECKED	Users review and apply proposals; Admins score failures and create correction/evaluation cases. Evidence: frontend/src/components/DeveloperPanel.jsx
Parallel execution	CHECKED	Independent background activity and realtime channels remain asynchronous; model notebook requests are intentionally serialized for GPU safety. Evidence: node-server/server.js; Untitled28_lightning_ai_bob_runtime.ipynb
Multi-agent design	CHECKED	Planner, coder and reviewer have distinct responsibilities and linked handoff artifacts. Evidence: bob_core/model_service.py

Metric coverage: Workflow completion, handoff accuracy and node-latency inputs are now stored; labeled quality baselines remain the next measurement step.

3. Practical Agent Integration

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Every tool documented	CHECKED	Every registered capability receives a non-empty description; contract tests fail when documentation is missing. Evidence: capabilities.py; scripts/verify-api-contract.mjs
Input schema	CHECKED	FastMCP generates typed input schemas and the server validates ownership and high-risk approvals. Evidence: mcp_server.py; node-server/server.js
Output schema	CHECKED	Tools return structured JSON and gateway responses consistently use ok/data or ok/error envelopes. Evidence: mcp_server.py; node-server/server.js
Retry	CHECKED	The external Colab tool boundary retries only transient failures and preserves exact transport attempts. Evidence: bob_core/colab_adapter.py
Timeout	CHECKED	Model and execution calls retain bounded timeouts; long model stages do not inherit a short gateway timeout. Evidence: bob_core/colab_adapter.py; node-server/config.js
Authentication	CHECKED	REST, MCP gateway, Socket.IO, LSP and terminal application access require a valid server session. Evidence: node-server/server.js; lsp.js; terminal.js
Cost	CAN ADD	Model token cost can be calculated from configured pricing, but generalized per-tool and infrastructure cost attribution is not implemented. Evidence: bob_core/model_config.py; node-server/dev-store.js
Latency	CHECKED	Gateway tools and model stages record start/end timestamps, durations and attempt counts. Evidence: node-server/server.js; bob_core/model_service.py
Security	CHECKED	Ownership, path containment, CSRF, RBAC, one-time approvals and input validation protect application tools. Evidence: node-server/server.js; auth-store.js

Metric coverage: API success, retry recovery, duration and error-rate inputs are available. Semantic argument-accuracy scoring still belongs in the evaluation dataset.

4. Retrieval-Augmented Generation

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Chunking	NOT NEEDED	The current agent uses selected workspace files rather than document chunk retrieval.
Metadata	NOT NEEDED	No retrieval index exists; context metadata is used for diagnostics, not semantic retrieval.
Embedding	NOT NEEDED	No embedding model is required for the current staged coding workflow.
Vector database	NOT NEEDED	A vector store is unnecessary at the current repository scale and product scope.
Citation	NOT NEEDED	The workflow produces code proposals rather than evidence-linked RAG answers.
Source display	NOT NEEDED	The IDE already displays selected files and diffs, not retrieved document sources.
Hybrid search	NOT NEEDED	Hybrid semantic/keyword retrieval is only useful after an indexed corpus is introduced.
Re-ranking	NOT NEEDED	No retrieval candidate list exists to re-rank.

Metric coverage: RAG metrics remain out of scope until Bob gains large-corpus repository or document question-answering.

5. Structured Outputs

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
JSON output	CHECKED	MCP, gateway, run, DLQ, correction, evaluation and notebook responses are JSON.
Validation	CHECKED	Invalid status, score, model, usage, header and configuration values are rejected.
Pydantic model	CHECKED	Pydantic v2 models validate plan, code, review, usage, error, DLQ, correction and evaluation shapes. Evidence: bob_core/contracts.py
Required fields	CHECKED	Authentication, approvals, corrections and human evaluations enforce required values server-side. Evidence: node-server/auth-store.js; dev-store.js
Error messages	CHECKED	Clients receive safe, understandable errors while sensitive technical details remain out of HTTP responses.

Metric coverage: Contract tests cover the complete tool catalogue; additional malformed external-model fixtures can deepen field-accuracy measurement.

6. Classification Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Confusion matrix	NOT NEEDED	Bob IDE does not expose a classification model.
Accuracy	NOT NEEDED	There is no labeled classification outcome; use task-success measures instead.
Precision	NOT NEEDED	There is no positive-class decision to score.
Recall	NOT NEEDED	There is no positive-class decision to score.
F1	NOT NEEDED	There is no classifier label set.
Macro average	NOT NEEDED	No multi-class classification task exists.
Weighted average	NOT NEEDED	No multi-class classification task exists.

Metric coverage: If an intent, safety or risk classifier is later introduced, add labeled per-class metrics based on failure cost.

7. Agent Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Tool selection	CAN ADD	Tool calls are logged, but the 12-case dataset does not yet label the expected tool for every scenario.
Tool arguments	CAN ADD	Schemas validate shape; semantic argument correctness still needs expected-value assertions.
Planning	CAN ADD	Offline cases validate deterministic plan contracts, but plan quality is not yet scored against gold steps.
Memory	CAN ADD	Workflow memory persists, but useful recall versus unnecessary context is not benchmarked.
Hallucination	CAN ADD	No automated unsupported-claim or invented-file detector is present.
Grounding	CAN ADD	Human groundedness scores exist, but automatic evidence linkage is not yet evaluated.
Task success	CAN ADD	Software checks and 12 offline cases pass, but representative executable end-to-end coding tasks are still needed.
Human approval	CHECKED	Protected force actions cannot bypass server-issued, user-, workspace-, operation- and target-bound approval tokens.

Metric coverage: The evaluation foundation is present. The next batch should turn the 12 cases into quality assertions for tools, plans, generated changes and executable outcomes.

8. Human Evaluation

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Correctness	CHECKED	Admins assign a required integer score from 1 to 5 and revisions are preserved.
Helpfulness	CHECKED	The Admin evaluation rubric stores a 1-5 helpfulness score.
Completeness	CHECKED	The Admin evaluation rubric stores a 1-5 completeness score.
Safety	CHECKED	The rubric stores safety, severity, failure category and expected behavior.
Tone	CAN ADD	Tone is not part of the current five-dimension rubric; add it if chat quality becomes a priority.
Groundedness	CHECKED	The rubric stores a required 1-5 groundedness score.
Citation quality	NOT NEEDED	Formal citations are not part of the current coding-proposal workflow.

Metric coverage: Evaluations require verdict, category, severity, notes and expected behavior. Revisions remain append-only within the entity.

9. Debugging

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Trace	CHECKED	Request ID, run/trace ID and user identity propagate through gateway model calls, run records and notebook responses.
Prompt	CHECKED	Redacted original prompts, corrected prompts and prompt-set versions are stored for quality review.
Tool logs	CHECKED	Tool name, argument field names/sizes, identity, result, duration, request/trace IDs and retry count are logged.
Token logs	CHECKED	Input, output and total token usage is captured when the runtime/provider returns it.
Error logs	CHECKED	Typed errors include component, stage, time, request context, retryability and attempt history.
Stack trace	CAN ADD	Safe errors are returned and redacted messages are logged; a restricted server-only traceback store is intentionally not yet implemented.
Root cause	CHECKED	Admin DLQ review requires a root-cause category and correction/dismissal remains auditable.

Metric coverage: The supported IDE path can now be diagnosed from request through model stages and DLQ. Sensitive tracebacks remain a deliberate follow-up control.

10. Observability

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Prompt logs	CHECKED	Redacted prompt records include prompt/model versions and linked run, DLQ, review and evaluation IDs.
Tool logs	CHECKED	A rotating JSONL event stream records redacted request and tool activity.
Token usage	CHECKED	Stage and aggregate input/output/total tokens are persisted.
Latency	CHECKED	Tool and stage durations feed the Developer Panel overview.
Errors	CHECKED	Error frequency, typed errors, retries, failed reviews and open DLQ items are visible.
Cost	CHECKED	USD estimates are calculated when input/output token prices are configured.
User feedback	CHECKED	Apply, discard and force-apply decisions are persisted as accepted, rejected or force-applied feedback.

Metric coverage: The Admin dashboard provides local/demo operational visibility. P50/P95/P99, alerting and external uptime monitoring remain production enhancements.

11. LLMOps

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Prompt version	CHECKED	Every model stage can carry BOB_PROMPT_SET_VERSION and stores the effective version.
Dataset version	CHECKED	The 12 deterministic cases use a versioned dataset manifest and stable IDs. Evidence: evals/cases.json
Model version	CHECKED	Provider, model ID, model revision and runtime contract version are recorded.
Evaluation pipeline	CHECKED	npm test runs software tests, tool contracts, notebook validation, 12 offline evaluations, lint and build.
A/B testing	CAN ADD	No paired prompt/model comparison or traffic split is implemented.
Rollback	CAN ADD	Git can restore source, but there is no one-command release, model or prompt rollback workflow.
Monitoring	CAN ADD	The Developer Panel tracks local events, but continuous quality sampling, alert thresholds and external monitoring are not implemented.

Metric coverage: Regression inputs, acceptance feedback and failure telemetry now exist. A/B comparison, quality thresholds and release rollback are the next LLMOps layer.

12. Cloud Deployment

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Docker	CAN ADD	No Dockerfile or compose/orchestration package exists.
API	CHECKED	Node REST/Socket.IO, Python MCP and staged Lightning/Colab HTTP endpoints expose core functionality.
HTTPS	CHECKED	The public notebook runtime uses an HTTPS ngrok tunnel; local services remain loopback HTTP.
Secrets	CHECKED	Passwords are hashed and the Colab bearer token is environment-only; notebook output and pasted secret cells were removed.
Load balancer	NOT NEEDED	A single trusted local/demo instance does not require traffic distribution.
Autoscaling	NOT NEEDED	Autoscaling is unnecessary until hosted demand and service objectives justify it.
Monitoring	CAN ADD	Application metrics exist, but no cloud resource monitoring or alert service consumes them.
Logging	CHECKED	Runtime and deployment-facing events are retained in rotating structured JSONL for the local deployment boundary.

Metric coverage: The notebook is suitable for a supervised Lightning/Colab demo. Containers, durable shared storage and cloud monitoring are still required for production hosting.

13. Privacy, Security and Responsible AI

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Authentication	CHECKED	First-run Admin setup, bcrypt cost 12, opaque server-side sessions, HttpOnly SameSite=Strict cookies and login throttling are implemented.
Authorization	CHECKED	Every non-public gateway route and realtime workspace join enforces identity and workspace ownership.
PII detection	CAN ADD	Secrets are redacted, but configurable PII detection for names, contact details and identifiers is not implemented.
Encryption	CAN ADD	HTTPS protects the tunnel, but local JSON state is not encrypted at rest.
Secret management	CHECKED	The Colab bearer is accepted only from BOB_COLAB_TOKEN; password/session secrets are hashed and never returned.
RBAC	CHECKED	Admin and User roles are enforced server-side for developer operations and hidden appropriately in the UI.
Human approval	CHECKED	High-risk force actions require a reason, explicit confirmation and a one-time server approval.
Audit logs	CHECKED	Authentication, user changes, ownership, approvals and developer actions are identity-linked in rotating append-only JSONL.

Metric coverage: The result is a strong trusted local/demo boundary, not hardened remote multi-tenancy. The integrated OS terminal remains outside tenant isolation.

14. Production Readiness

This consolidates the source checklist's architecture, AI, evaluation, debugging, deployment, security, reliability, cost and documentation review.

CHECKLIST ITEM	STATUS	PROJECT FINDING / EVIDENCE OR ACTION
Architecture - Diagram	CHECKED	README documents the frontend, Node gateway, Python MCP and Lightning/Colab runtime flow.
Architecture - Components	CHECKED	Code and docs separate UI, authentication/gateway, capability, model and persistence responsibilities.
Architecture - Workflow	CHECKED	Plan -> select/replan -> code -> review -> proposal -> apply/DLQ/evaluation is implemented.
AI - Agent	CHECKED	Bob provides supervised agentic coding inside a reviewable IDE workflow.
AI - Planner	CHECKED	Planning and replanning stages are explicit and persisted.
AI - Tools	CHECKED	A documented MCP capability registry exposes deterministic actions.
AI - Memory	CHECKED	Workflow, identity, ownership and quality records persist in versioned JSON.
AI - RAG	NOT NEEDED	RAG is unnecessary for the current selected-file coding workflow.
Evaluation - Dataset	CHECKED	Twelve versioned normal, edge, failure and adversarial cases exist.
Evaluation - Metrics	CHECKED	Offline pass rate plus human rubric, error, retry, latency, token, cost and feedback measures are available.
Evaluation - Human evaluation	CHECKED	Admin evaluations preserve 1-5 rubric revisions and verdict metadata.
Debugging - Logs	CHECKED	Redacted request/tool/model/auth/audit events are persisted.
Debugging - Traces	CHECKED	Request, trace/run and user IDs link gateway, model and notebook stages.
Debugging - Errors	CHECKED	Structured errors and immutable DLQ attempt histories are retained.
Deployment - Docker	CAN ADD	No reproducible container package exists.
Deployment - Cloud	CHECKED	Lightning/Colab plus authenticated HTTPS ngrok is implemented for supervised model compute.
Deployment - Monitoring	CAN ADD	Local operational metrics exist; cloud resource monitoring and alerting do not.
Security - Authentication	CHECKED	State-safe user login, setup, sessions, CSRF and logout are implemented.
Security - Authorization	CHECKED	RBAC and workspace ownership are enforced at gateway and realtime boundaries.
Security - Secrets	CHECKED	Passwords/sessions are hashed and BOB_COLAB_TOKEN is environment-only.
Security - Encryption	CAN ADD	At-rest encryption and a managed key lifecycle are not implemented.
Reliability - Retry	CHECKED	Transient Colab calls retry exactly three times before DLQ.
Reliability - Timeout	CHECKED	Model and command boundaries use bounded timeouts.
Reliability - Fallback	CHECKED	Safe unconfigured responses, chat fallback and recoverable proposal/Git flows remain available.
Reliability - Cache	CAN ADD	No deterministic result cache exists; add only after invalidation and privacy rules are defined.
Cost - Tokens	CHECKED	Input/output/total tokens are captured by the v4 notebook and local adapter.
Cost - Latency	CHECKED	Stage/request/tool duration is recorded and summarized.
Cost - Model routing	CAN ADD	A single configured model endpoint is used; routing needs comparative evaluation first.
Cost - Cache	CAN ADD	No computation cache is implemented.
Documentation - README	CHECKED	Comprehensive setup, operation and troubleshooting documentation exists.
Documentation - API	CHECKED	MCP catalogue, schemas, routes and notebook capabilities are exposed and contract-tested.
Documentation - Architecture	CHECKED	Migration, integration and handoff documents explain system decisions.
Documentation - Demo	CHECKED	Sample workspaces, first-run setup UI and notebook run instructions support demonstrations.
Documentation - Future work	CHECKED	Explicit scope limitations and follow-up controls are documented in the audit and project docs.

Production AI design review

1. Why does this need an LLM?

The LLM interprets open-ended coding requests, plans multi-file changes, generates code and critiques it. Identity, ownership, schemas, path safety, retries, approval and Git staging remain deterministic.

2. What decisions are delegated?

Delegated: plan content, generated code and reviewer judgment. Deterministic: access control, workspace ownership, retry policy, validation, proposal state, approval consumption and file/Git mutation.

3. Five likely failure modes

Incorrect plan; missing context; unsafe or malformed code; Lightning/ngrok outage; and a reviewer-failed proposal being forced by a user. Each now has a visible control or audit record.

4. How are failures detected?

Pydantic validation, structured errors, reviewer PASS/FAIL, hash conflicts, retry exhaustion, DLQ creation, feedback, health endpoints and Admin quality records.

5. How does the system recover?

Replan, safe fallback, three transient attempts, DLQ classification/correction, proposal discard, Git restore and explicit user-owned Force Apply and Stage.

6. How do we know a version is better?

The versioned dataset, human revisions, feedback and telemetry are now a baseline. Gold task/tool assertions and paired A/B evaluation are still needed for defensible model-quality comparison.

7. How are data and secrets protected?

The app uses sessions, RBAC, ownership, CSRF, redaction and environment-only bearer configuration. PII detection and encryption at rest remain open.

8. Cost per successful task

Token cost can now be estimated when prices are configured and acceptance feedback is stored. A report joining cost to accepted executable task success remains addable.

9. What breaks from 10 to 1 million users?

Local JSON, single-process Node/Python services, the serialized GPU runtime, ngrok, provider quotas and lack of load balancing, autoscaling, databases and distributed queues.

10. Would a customer trust it?

For a supervised local/demo workflow, substantially more than before. For remote production, Docker/cloud operations, PII controls, encryption, alerts, backups and hardened OS isolation remain required.

Recommended next additions

1. Agent-quality evaluation

Expand the 12 cases into gold tool selections, argument assertions, plan-quality anchors and executable code acceptance tests.

2. Privacy and encrypted persistence

Add configurable PII detection, data retention/deletion rules and encryption-at-rest key management.

3. Production monitoring

Add percentile metrics, health/resource collection, alert thresholds, uptime checks and incident runbooks.

4. Reproducible deployment

Containerize the frontend/gateway/MCP services, define durable storage/backups and add CI/CD plus rollback.

5. Comparative LLMOps

Add paired prompt/model experiments, regression thresholds and cost-per-accepted-task reporting before model routing.

Scope decisions

RAG, classification metrics, load balancing and autoscaling are not impossible; they remain not needed for the current selected-file, trusted local/demo IDE. RBAC is no longer out of scope because Admin and User roles are now implemented. The integrated terminal remains explicitly outside hardened multi-tenant OS isolation.